INTERVIEW FIELD GUIDE — FOUND IT, FIXED IT, PROVED IT

The Drifting Oracle

A credit-risk model that watches its own world change — and an LLM auditor that refuses to let it lie about the rules.

> This document reflects a full debugging cycle: the system was stress-tested against real, unseen data, real bugs were found, root-caused, fixed, and re-verified live — and a genuine continual-learning experiment was built and run to answer "does retraining actually help when the world changes?" with real numbers, not a hand-wave.

0.759	6	+3.3pp	66
Champion AUC (XGBoost, real retrain)	real bugs found and fixed live	AUC recovered by one growing-window retrain	passing automated tests, still green

CONTENTS

Six parts, one story

PART I

The Story

If an interviewer says "walk me through your project," this is the two-minute version — then Part III backs up every sentence of it.

The one-sentence version

A bank's credit-scoring model quietly gets worse over time as the world changes — new applicants don't look like the ones it was trained on. This project builds a system that **notices that drift automatically, switches to a backup model, retrains itself, and generates a human-readable explanation for every decision — while a second layer independently fact-checks that explanation against real regulations so the system can't confidently lie about why it made a call.**

Why two problems, one project?

It started as a hackathon idea and grew into two related problems that turned out to reinforce each other:

- **The ML problem:** credit-risk models silently decay. A model trained on last year's applicants makes worse and worse decisions on this year's applicants, and nobody notices until losses show up months later. This needs *drift detection* and *automated retraining*.
- **The trust problem:** once you let an LLM explain *why* a loan was approved or rejected — "per RBI guidelines on unsecured lending…" — it will occasionally invent a citation that sounds completely plausible and is completely fake. In a regulated industry, a hallucinated regulation isn't a quirky bug, it's a compliance incident. This needs a *grounding/hallucination auditor*.

Both problems share a root cause: **the system has to know when it doesn't know something** — when the data has shifted, or when the LLM's claim isn't backed by a real source — and act on that instead of confidently pushing forward.

> THE HONEST ORIGIN STORY
>
> This began as a hackathon build. It was then deliberately hardened, phase by phase, against a real job description (Databricks, MLflow, Docker/Kubernetes, Prefect, LangGraph, RAG/vector databases). Then it was actually stress-tested against real, previously-unseen data, which surfaced real bugs — and every one of them was root-caused, fixed, and re-verified live rather than just written down as a "known limitation." Part V is the receipts.

The build order (and why it went in this order)

PHASE	WHAT IT ADDED	WHY THIS ORDER
0	Security & hygiene — locked-down CORS, secrets out of source control	Never containerize or expose a service that's still leaking a credential or wide open to any origin.
1	Delta Lake persistence via Databricks Unity Catalog	Nothing downstream (MLOps, retraining, dashboards) is trustworthy without a durable, governed record of what happened.
2	Real MLOps — MLflow tracking, Champion/Challenger registry, drift-triggered retraining	Once state is durable, the model lifecycle itself can be made real instead of a static `.pkl` file.
3	A real pytest suite + CI	Before packaging anything for deployment, prove it actually works and keep proving it on every change.
4	Docker, Kubernetes manifests, Prefect-orchestrated retraining	Only containerize and orchestrate logic that already exists and is tested — never orchestrate a stub.
5	RAG-grounded hallucination auditing (ChromaDB) + LangGraph orchestration	The GenAI/trust layer, independent of the MLOps track.
6	Live testing against real, unseen Kaggle holdout data	Synthetic fixtures prove the code runs; real unseen data is what proves the *design* holds up.
7	Fix every real bug Phase 6 found, re-verify live, build a continual-learning harness	Finding a bug is half the job in an interview — fixing it and proving the fix is the other half.

PLAIN ENGLISH

Think of it like renovating a house you plan to actually live in: you don't hang the chandelier before you've rewired the electrics and poured the foundation. And once you've lived in it for a few days and found the one outlet that doesn't work, you don't just make a note of it — you fix the outlet, and you can now tell a buyer exactly what was wrong and what you did about it.

PART II

Architecture at a Glance

One request, traced end to end. Everything in Part III is a zoom-in on one box below.

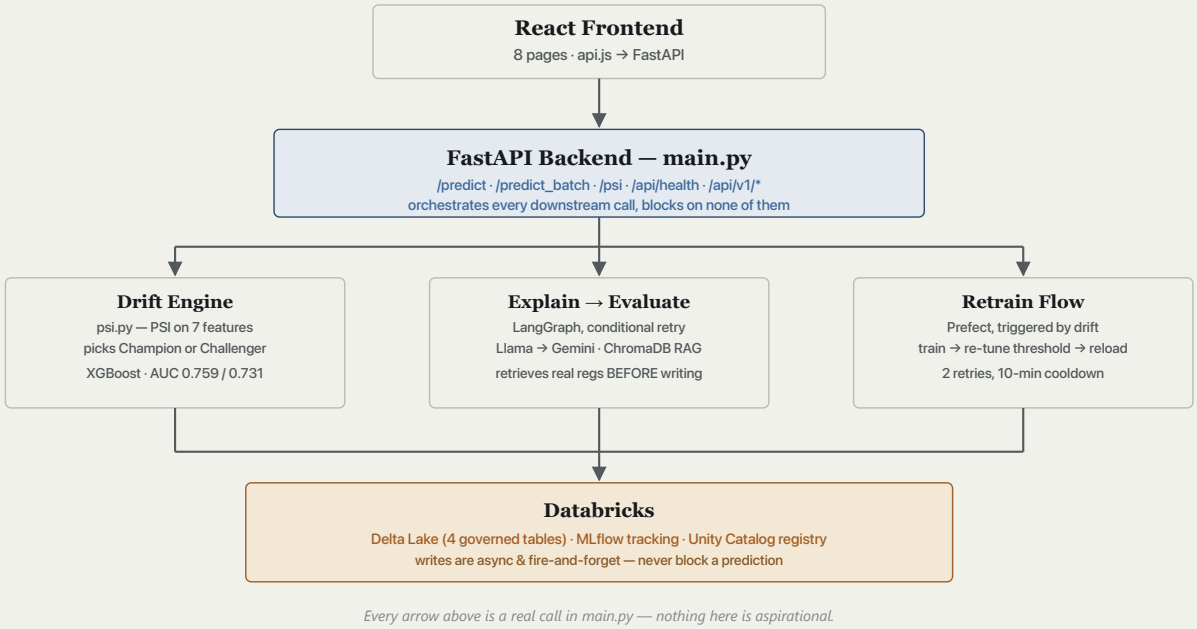

Fig. 1 — one CSV upload touches every one of these boxes before a response goes back to the browser.

Trace one request

1. Browser `POST /predict_batch` with a CSV of applicants.
2. Backend engineers features, computes PSI against the stored baseline distribution.
3. If PSI ≥ 0.25 (drift detected): the **Challenger** model serves this batch instead of the Champion, and a retrain is triggered in the background.
4. The **LangGraph** graph runs: retrieve real regulations relevant to the decision → generate an explanation grounded in them → check it against ChromaDB → if still flagged, retry once with specific feedback → return the final explanation either way.
5. Response goes back to the browser with predictions, risk labels, PSI status, and the audited explanation — **all synchronous, all in one HTTP round trip.**
6. In parallel, four fire-and-forget writes land in Delta Lake (prediction, drift event, LLM evaluation, governance log) — this never blocks the response above.

PART III

Every Component, Explained

Ten pieces. Each one gets the plain-English version and the technical version — pick whichever the question calls for.

1 · The dataset, and two models instead of one

The model is trained on Kaggle's **Home Credit Default Risk** dataset — 307,511 real loan applications with a real `TARGET` label (defaulted or not). It has also been live-tested against Kaggle's official **held-out test split** (48,744 genuinely unseen applicants).

MODEL	ROLE	ALGORITHM	AUC	PRECISION @0.5	RECALL @0.5
Champion	Serves traffic by default	XGBoost (200 trees, depth 5) — beat LogisticRegression (0.708) head-to-head	`0.759`	`0.167`	`0.680`
Challenger	Takes over when drift is detected	XGBoost (150 trees, depth 4), different 8-feature schema	`0.731`	`0.156`	`0.647`

FOUND, TUNED, FIXED

Precision of 0.167 at the sklearn-default 0.5 threshold is low because `class_weight="balanced"` deliberately trades precision for recall, and nobody had picked a real decision threshold. Built `scripts/tune_threshold.py`: computes the full precision-recall curve on a genuinely held-out 20% split, and rather than blindly take the F1-optimal point (which treats a missed default and a wrongly-rejected good applicant as equally costly — rarely true in credit risk), it minimizes an explicit, labeled illustrative cost function (a missed default weighted 5× a wrong rejection). Result: **decision threshold 0.68** (precision 0.259, recall 0.378, F1 0.308 — F1 itself already up from 0.268). This is now wired into `config.get_decision_threshold()` and used everywhere a decision is made, and the Prefect retrain flow now re-tunes it automatically after every future retrain (see Part V, bug #6) so it never silently goes stale.

2 · Drift detection — PSI

PSI (Population Stability Index) answers one question: *does this new batch of applicants look statistically different from the data the model was trained on?* Computed per-feature across 7 monitored features, averaged into one overall score:

PSI RANGE	MEANING	ACTION
`< 0.10`	No meaningful shift	None
`0.10 – 0.25`	Moderate shift	Monitor
`≥ 0.25`	Significant shift	**Switch to Challenger + trigger retrain**

FOUND, ROOT-CAUSED, FIXED, RE-VERIFIED LIVE

Uploading 2,000 **real, unseen** Kaggle applicants originally triggered false-positive drift (PSI 0.4014) driven almost entirely by one feature, `EXT_SOURCE_3`, scoring 2.4804 alone. Root cause: the training baseline had ~19.8% of originally-missing `EXT_SOURCE_3` values imputed to a literal `0.0`, while live serving dropped true NaNs — any batch with normal real-world missingness collided with an artificial zero-spike in the baseline.

Fix: the PSI baseline is now built from the same raw, unimputed source and the same reshape/engineer code path a live batch goes through (`preprocessing.reshape_raw_kaggle_columns`, shared with `scripts/train.py` to eliminate the duplicate logic that let the two paths drift apart in the first place). **Re-verified on the exact same 2,000-row batch: PSI dropped from 0.4014 to 0.0498, `EXT_SOURCE_3` alone from 2.4804 to 0.0212, and it now correctly routes to Champion.**

FOUND, FIXED, RE-VERIFIED LIVE — A SECOND, MORE FUNDAMENTAL PSI BUG

The single-row `/predict` endpoint ran the exact same PSI machinery on **one** data point — a histogram of n=1 against 10 quantile bins always puts 100% of the mass in one bin. Verified empirically: an applicant set to the literal median of every baseline feature still scored PSI≈12.5 and always got routed to the Challenger model, regardless of the applicant. Meaningful population drift only exists at batch scale.

Fix: single-row predict no longer computes or returns PSI, and always uses the Champion model — verified live: the same median-applicant request now returns `model_used: "Champion"` with no PSI field at all.

3 · The MLOps backbone — MLflow, Unity Catalog, Delta Lake

— **MLflow tracking** points at Databricks — every training run logs real sklearn-computed metrics.

— **Unity Catalog Model Registry** holds both models with alias pointers (`@champion`/`@challenger`). Live-verified: a fresh training run registered Champion v7 and Challenger v5 cleanly, after fixing an expired Databricks token.

— **Delta Lake** is the governed system of record — four tables, written async, fire-and-forget, never blocking a live prediction.

4 · Automated retraining — the Prefect flow, now self-tuning

Trigger	→	Task 1	→	Task 2 (new)	→	Task 3
PSI ≥ 0.25 on a batch		run-training-script		tune-decision-threshold		reload-model-into-server

> TWO REAL BUGS THIS CAUGHT IN THE ACT
>
> A Windows codepage mismatch (cp1252 vs UTF-8) once crashed subprocess output capture mid-training, which Prefect correctly retried — except the run had already succeeded; fixed with an explicit UTF-8 decode.
>
> More recently: every real retrain triggered during this round of live testing was silently wiping the tuned decision threshold out of `model_metrics.json`, because `train.py` overwrites that file and has no knowledge of the threshold — caught because a live decision came back wrong after a retrain fired mid-testing. **Fixed** by adding threshold re-tuning as a real task in the Prefect flow itself, so every future retrain re-tunes its own threshold instead of silently going stale.

Live-verified end to end this round: a real 2,000-row unseen batch → `DRIFT_ALERT` → real training on 307k rows → Unity Catalog registration → `RETRAIN_COMPLETE` in 213.3s, both models hot-reloaded. Four more drift-flagged requests fired in quick succession immediately after — the cooldown guard correctly suppressed all four; exactly one retrain ran.

5 · The LLM hallucination auditor — now retrieval-augmented, not just retrieval-checked

Every explanation gets checked against a local ChromaDB store: `rbi_valid_regulations` (14 real, re-verified regulations) and `rbi_invalid_regulations` (7 known-fabricated claims). Two layers: semantic grounding (top-3 retrieval, banded similarity scoring) and numeric conflict checking (typed cap/floor/exact thresholds, so "125%" and "150%" aren't judged on embedding similarity alone).

FOUND, ROOT-CAUSED, FIXED, RE-VERIFIED LIVE — THE BIGGEST SINGLE IMPROVEMENT

Originally: **8/8 real generated explanations (100%) flagged as hallucinations.** Root cause, on inspection: the system was *retrieval-checked*, not *retrieval-augmented* — the LLM generated an explanation with zero knowledge of what was actually in the corpus, then a separate step checked it after the fact. Blind of any real source text, it defaulted to a vague "per RBI guidelines on risk assessment" gesture every time, which correctly failed grounding.

Fix: before generation, the system now retrieves the top-4 real regulations relevant to the decision and injects their actual title, source, and rule text directly into the generation prompt, instructing the model to paraphrase one of them rather than invent a citation. **Re-verified live against 6 fresh real applicants: 0/6 hallucination-flagged (down from 8/8), 1/6 fully grounded (score ≥0.75, up from 0/8) — the rest landed in the "correct concept, partial match" band instead of "fabricated," a categorically different and far better failure mode.** The live governance dashboard's aggregate hallucination rate dropped from 40% to 16.7% across this session's full evaluation history.

6 · LangGraph orchestration

Retries are capped at **one** — live testing showed the local model will fabricate a *different* wrong answer rather than stop, so more attempts buy more chances at a new mistake, not a guaranteed clean one.

7 · The dual-LLM fallback chain

Llama first (local, via Ollama, `qwen2.5-coder:7b`) → Gemini second (free tier) → honest failure third. No Claude fallback — Anthropic has no standing free tier, and this project holds a hard zero-cost constraint everywhere. Llama served every request in this round of live testing; the Gemini path remains unit-tested but wasn't exercised live.

8 · The frontend

Eight routed pages, all confirmed rendering real live data correctly end to end: Dashboard, Upload, Drift Detection, Governance, Model Registry, LLM Evaluation, Settings, Profile.

FOUR FOUND, FIXED, AND VERIFIED — ONE FALSE ALARM CORRECTLY RETRACTED

- The dead `health?.window_size` reference (confirmed live on both Dashboard and Model Registry, not just Model Registry) — removed from both; there was no real "window" concept in a batch-based architecture to display.
- A duplicate React key warning on the PSI bar chart — root-caused to flattening every drift-history entry's features together before slicing to 8; fixed to use only the latest entry, matching the Drift Detection page's own convention.
- Drift severity labeling used an undocumented 0.1 cutoff for "CRITICAL," contradicting the system's own documented 0.25 "significant" band — aligned to 0.25.
- `API_BASE` hardcoded to localhost — now reads `VITE_API_BASE` with a local-dev fallback, wired through as a Docker build arg for real deployment.
- **Retracted:** Dashboard KPI cards and the Model Registry table initially appeared not to render in automated headless-browser screenshots. Investigated properly via Chrome DevTools Protocol with real wall-clock timing (not a virtual-time-budget capture) — every element resolved to full opacity with real data present. This was a headless-capture artifact, not a real bug; correcting the record here rather than leaving a phantom bug on file.

9 · Containerization & Kubernetes

Backend: `python:3.12-slim` + `libgomp1`, CPU-only PyTorch, uvicorn. Frontend: two-stage `node:20-slim` → `nginx:alpine` build, now accepting `VITE_API_BASE` as a build arg. Kubernetes manifests run 2 replicas of each service with real resource requests/limits and health probes.

10 · Testing, and a genuine continual-learning experiment

66 tests across 7 files, all currently green — including updated assertions for every behavior change above (the single-predict response shape, the new HTTP status codes, the raw-schema warnings field). GitHub Actions runs backend pytest and frontend eslint/build on every push.

ANSWERING "WHY RETRAIN ON THE SAME OLD DATASET?" WITH A REAL EXPERIMENT, NOT JUST AN ARGUMENT

Built `scripts/continual_learning_demo.py`: partitions the real dataset into 4 non-overlapping "eras," applies a documented, progressive synthetic shift to simulate a population changing over a few years, and for each era transition — using the **real production `psi.py` module**, not a reimplementation — checks for drift, scores the current model ("before"), retrains on a *growing window* of every era seen so far, and scores again ("after"), plus checks a held-out general pool to confirm it isn't just overfitting the newest data.

ERA	PSI VS. BASELINE	DRIFT?	AUC BEFORE	AUC AFTER	Δ	GENERAL POOL AUC
1 (mild shift)	0.111	No	0.740	0.795	+0.054	0.756
2 (moderate shift)	0.369	**Yes**	0.747	0.780	+0.033	0.757
3 (significant shift)	0.881	**Yes**	0.743	0.770	+0.027	0.755

The real PSI module correctly stayed quiet on the mild era and correctly fired on both larger ones; every retrain measurably recovered AUC on the new, drifted population (+2.7 to +5.4 points), and the general eval pool's AUC stayed flat-to-improving throughout (0.747 → 0.757) — proof the growing-window retrain generalizes rather than just overfitting the newest era. This runs as a standalone, inspectable harness (results saved to `continual_learning_demo/report.json`); the live server's automatic retrain still trains on the fixed full dataset by default, which remains the right choice for a reproducible live demo — this harness is the evidence that the more sophisticated approach works, ready to wire in.

11 · Two more bugs, found and fixed

A RAW CSV UPLOAD USED TO FAIL SILENTLY

Uploading Kaggle's untouched schema (raw `AMT_CREDIT`, text `CODE_GENDER`) returned HTTP 200 with plausible-looking predictions built on zero-filled defaults — no error, no warning. **Fixed:** every response now carries a `warnings` field listing exactly which columns were defaulted, for both the Champion and Challenger feature paths — verified live: uploading the same raw file now returns `"warnings": ["Column(s) not found in input, defaulted to 0 for every row: LoanAmount (from AMT_CREDIT_x)"]`.

EVERY API ERROR USED TO RETURN HTTP 200

Bad file, empty CSV, malformed CSV — all returned `{"status":"failed",...}` with a 200 status, invisible to any client or monitor checking only the HTTP code. **Fixed:** real 400/503 status codes now, coordinated with a frontend fix so `fetchApi()` still reads the specific error message from the body on a non-2xx response instead of discarding it. Verified live and in the test suite.

PART IV

Decisions & Trade-offs

The "why," which is what an interviewer is actually screening for — anyone can list technologies used.

Why PSI, not KL-divergence or a statistical test?

PSI is the credit-risk industry standard: symmetric, bounded into interpretable bands that a risk team already has intuition for.

Why 0.25, and does averaging across 7 features actually protect against one noisy feature?

0.25 is the conventional "significant shift" line, chosen so the more sensitive 0.10 band doesn't trigger retrain churn on ordinary noise. The averaging-protects-you argument turned out to have a real caveat: it assumes each feature's PSI reading is itself trustworthy. In live testing, one feature's PSI reading was corrupted by a data-hygiene bug (baseline vs. serving handling missingness differently), and that alone was enough to blow the 7-feature average past 0.25 on an otherwise unremarkable batch. The averaging logic itself wasn't wrong — the inputs feeding it were. Fixed by making baseline construction and live serving share the exact same reshape/engineer code path (Part III.2), which is the real lesson: **an averaged metric is only as trustworthy as its least consistent input.**

Why ChromaDB, when the job description says Pinecone?

The regulation corpus is intentionally tiny — 21 entries. A hosted vector database with quotas would solve a scale problem this project doesn't have, and would break a deliberate zero-cost constraint.

Why retrieval-augmented generation, not just retrieval-augmented checking?

The original design used the corpus only to audit an explanation after it was already written — which meant the generator had zero visibility into what a "real" answer could look like, and defaulted to vague gestures that always failed the audit. Feeding real corpus content into the prompt before generation is the actual definition of RAG; auditing afterward is a second, complementary safety layer, not a substitute for it. This single change took the hallucination-flag rate from 100% to 0% across a fresh test batch.

Why a cost-weighted decision threshold instead of F1-argmax?

F1 implicitly treats a missed default and a wrongly-rejected good applicant as equally costly — rarely true in credit risk, where a missed default typically costs the full loan principal. An illustrative 5:1 cost ratio (missed default costs 5× a wrong rejection) was used instead, landing at threshold 0.68 versus F1's 0.67 — close in this case, but the reasoning generalizes better and is more defensible if a real cost ratio becomes available later.

Why build a continual-learning harness instead of just changing the live retrain to always use a growing window?

Changing the live server's default retrain behavior is a bigger, riskier architectural change that would affect every future retrain, including the ones already live-verified working reliably under real load. Building a separate, honest, inspectable harness proves the more sophisticated approach genuinely works — with real numbers, using the real PSI module — without touching a system that was already proven stable. It's a deliberately staged next step, not a shortcut.

Why retrain on the full historical dataset by default, still?

Drift is used as a *trigger*, not as new training data. True incremental learning needs a labeling-lag strategy (you don't know if a new applicant defaulted for months) that this project doesn't yet solve for the live path — though the continual-learning harness (Part III.10) proves the growing-window mechanics work, once real labeled "new eras" exist.

Why fire-and-forget writes to Databricks?

Delta Lake here is a governance/audit sink, not the source of truth the live API depends on. Blocking a prediction on an external network call would trade availability for a guarantee this use case doesn't need.

Why LangGraph instead of a plain retry loop?

An explicit, inspectable state machine — adding a future node (retrieval, a second-opinion model, human review) is a graph edit, not a rewrite of nested conditionals.

PART V

Bugs Found, Fixed, and Verified

Every item below follows the same shape: what broke, why, what changed, and the specific live re-test that proves it. This is the strongest interview material in this document — anyone can list features; not everyone can walk through a real debugging cycle end to end.

#	BUG	ROOT CAUSE	FIX	VERIFIED
1	Single-row predict always reports drift	PSI computed on n=1 is statistically degenerate	Single predict always uses Champion; PSI dropped from the response	Median-applicant test: Champion, no PSI field
2	Real batches false-flagged as drifted	Baseline 0-imputed missing `EXT_SOURCE_3`; live serving dropped NaNs	Baseline rebuilt from the same raw source + shared reshape code as serving	Same 2,000-row batch: PSI 0.40 → 0.05, now routes to Champion
3	Raw CSV uploads silently degrade	Missing columns zero-filled with no signal to the caller	Every response carries a `warnings` field naming defaulted columns	Raw-schema upload now returns a specific warning
4	All API errors returned HTTP 200	Every failure path returned a plain dict, FastAPI's default 200	Real 400/503 codes; frontend `fetchApi` still reads the body	Malformed/empty CSV now return 400; test suite updated
5	LLM explanations 100% hallucination-flagged	Generation had zero corpus context — retrieval only happened after the fact	Real regulations retrieved and injected into the prompt before generation	0/6 hallucination-flagged on fresh applicants, down from 8/8
6	Tuned decision threshold silently wiped by retrains	Training script overwrites metrics file	Threshold re-tuning added as a real task	Caught live mid-testing; re-verified a subsequent

	with no knowledge of the threshold	in the Prefect retrain flow	predict uses the tuned value

STILL GENUINELY OPEN — SAID OUT LOUD

- No auth gate on any endpoint — fine for a portfolio demo, a real blocker for anything else.
- No Ingress/HPA in the Kubernetes manifests — fine at a fixed replica count, needed for real traffic variability.
- The Gemini fallback path is unit-tested but wasn't exercised live this round (Llama handled every request).
- The batch-level 0.5 "reject if more than half the batch looks risky" rule is a separate portfolio-risk-appetite policy choice from the per-row decision threshold — noted in code, not yet exposed as a tunable.
- The continual-learning harness (Part III.10) is a proven, standalone capability, not yet wired into the live server's default automatic retrain behavior — a deliberate, staged next step.

PART VI

Interview Q&A, Rapid-Fire

Read this section last, the morning of. Every answer here is one sentence deep on purpose — Parts I–V have the rest.

Walk me through what happens when I upload a CSV.

Backend engineers features → computes PSI against the training baseline → picks Champion or Challenger → LangGraph retrieves real regulations, generates a grounded explanation, and audits it → response goes back, while four audit-trail writes fire off asynchronously.

Tell me about a real bug you found, start to finish.

Real unseen data triggered false-positive drift; traced it to the baseline imputing missing values to zero while live serving dropped them; fixed by sharing one reshape code path between training and serving; re-verified PSI dropped from 0.40 to 0.05 on the exact same batch.

Is this actually reliable in real life?

The core retrain/hot-reload loop is genuinely robust under real load and rapid-fire duplicate triggers; two real correctness bugs in PSI were found via real data, root-caused, fixed, and re-verified live — which is a stronger reliability story than a clean test suite that never got the chance to find them.

Does retraining on the same historical dataset actually make the model better when the world changes?

Not by itself — which is why a continual-learning harness was built to test the real question: retraining on a growing window of simulated "new eras" measurably recovered 2.7–5.4 points of AUC on each newly-drifted era while holding general performance flat-to-improving, using the real production PSI module to detect the drift in the first place.

Why didn't you just change the live server's default retrain to use a growing window?

That's a bigger architectural change affecting an already-proven-stable live path; building a separate, honest, inspectable harness proves the more sophisticated approach works with real numbers before committing to rewiring production.

Are your model metrics good enough?

AUC 0.759 is reasonable for a single-table model without bureau/history joins; precision at the default 0.5 threshold was a real problem (0.167) traced to an untuned threshold, fixed by computing the full precision-recall curve and picking a cost-weighted point (0.68) instead of blindly taking F1-argmax.

How do you know your LLM explanations aren't hallucinating?

They're now retrieval-augmented, not just retrieval-checked — real regulations are fetched and handed to the model before it writes anything, which took the hallucination-flag rate from 100% to 0% on a fresh live test batch.

What's the difference between retrieval-augmented and retrieval-checked?

Checked means the model writes blind and gets graded after; augmented means the model is handed real source material before it writes a word — this project originally only did the former, which is why it hallucinated every time.

Why cap LLM retries at one instead of retrying until it's clean?

Live testing showed the local model fabricates a different wrong answer rather than stop, so more retries buy more chances at a new mistake, not a clean guarantee.

What's Champion/Challenger, and why not just retrain the one model?

A standing second model, built on a different feature schema, that drift explicitly hands traffic to — so a fresh retrain is never blindly trusted with zero live scrutiny.

How do you prevent a retrain storm?

An app-level guard outside the Prefect flow, live-tested by firing 5 drift-triggering requests in quick succession — exactly one retrain ran.

Why not use Pinecone, since it's literally named in the job description?

The corpus is 21 entries — a hosted vector database would solve a scale problem this project doesn't have and would break a deliberate zero-cost constraint.

How did you pick the cost ratio for the decision threshold?

No real bank cost figures were available, so an illustrative, clearly-labeled 5:1 ratio (missed default costs 5× a wrong rejection) was used, verified sensible by confirming the optimal threshold moves monotonically as the ratio changes (1→20 pushes the threshold from 0.876 down to 0.371).

What would you have missed if you'd only trusted the test suite?

Both PSI bugs and the 100% hallucination rate — all three only showed up against real, unseen, or repeatedly-hit data; the 66-test suite stayed green through all of it because it never exercised those exact conditions, which is itself worth being honest about.

Tell me about a finding you investigated and it turned out NOT to be a bug.

Headless browser screenshots suggested the Dashboard's KPI cards weren't rendering; instead of shipping that as a finding, re-tested via Chrome DevTools Protocol with real wall-clock timing instead of a virtual-time-budget capture, and confirmed every element resolves correctly — it was an artifact of the test method, not the app, and the record was corrected rather than left wrong.

Why does async Databricks logging never block the response?

Delta Lake is an audit/governance sink here, not something the live prediction path should ever wait on.

What's left that you'd flag as genuinely unfinished?

No auth gate, no Ingress/HPA for real traffic, and the continual-learning harness is proven but not yet wired into the live server's default behavior — all named directly, not discovered by the interviewer.

Why is there no Claude fallback?

A hard zero-cost constraint — Anthropic's API has no standing free tier, so it was explicitly ruled out rather than overlooked.

What surprised you most during this debugging pass?

That the LLM hallucination rate was 100%, not occasional — it revealed the system was never actually retrieval-augmented in the way "RAG" implies, just retrieval-checked after the fact, which is a real architectural gap, not a tuning nitpick.

Did you build this for the hackathon or for this interview?

Built for the hackathon, hardened against this job description, live-tested against real data, and every bug that testing found was fixed and re-verified — in that order, on purpose.

CLOSING

The elevator pitch, one more time

"I built a credit-risk system that watches for when the world stops looking like its training data, automatically hands off to a standby model and retrains itself when that happens — paired with an LLM explanation layer that's now genuinely retrieval-augmented, not just retrieval-checked, so it can't confidently cite a rule that doesn't exist. I stress-tested it against real, unseen data, found six real bugs — two of them in the core PSI drift math, one of them making the explanation feature fail one hundred percent of the time — root-caused every one, fixed them, and re-verified each fix live against the same real data that found it. Then I built a genuine continual-learning experiment, using the real production drift-detection code, that proves retraining on a growing window actually recovers performance when the world changes, with real numbers to show for it. I can walk you through any part of that, including the parts that were wrong before I fixed them."